

Cappella Website

Animation Systems Report

Complete inventory of animations — where they are used, how they work, and their code.

Source file: Cappella Website.dc.html (single-file Figma replication)
Generated: 14 July 2026 · Branch: main

Overview

All animation code lives in `Cappella Website.dc.html`. CSS animations sit in the `<style>` block (lines 46–448); JavaScript-driven animations are implemented as `_setup*()` methods on the page class and wired up around lines 690–715. The site combines three engines:

Engine	Used for
Plain CSS transitions / keyframes	Scroll reveals, menu overlay, film grain, hover states, underline wipes
GSAP + ScrollTrigger + Lenis	Smooth scrolling, scroll-scrubbed parallax, curtain reveal, S-curve draw, split-text
Raw canvas / WebGL render loops	Water-flow dot grid, Bayer dithers, GLSL hover shader, logo marquee

Contents

- 01

Preloader logo sequence
- 02

Hero intro: grid lines + character wave
- 03

Hero headline masks + scroll-out
- 04

Water-flow dot grid
- 05

Text scramble (nav + portfolio)
- 06

Generic scroll reveals
- 07

Lenis smooth scroll + GSAP ticker
- 08

Hero parallax zoom
- 09

Line-split paragraph reveals
- 10

Odometer stat counters
- 11

Whitespace staircase reveal
- 12

Journey: S-curve + phase slider
- 13

Dither-resolve photo entries
- 14

Portfolio WebGL dither hover
- 15

Cappella Edge accordion
- 16

Footer curtain + staged rise + clock
- 17

Operators logo marquee
- 18

Menu overlay
- 19

Film grain
- Inactive systems & accessibility notes

01 — Preloader logo sequence

`_setupPreloader()` · line 1254

CSS keyframes · line 127

Where: Fires immediately on page load, covering the whole viewport.

How: A fixed red curtain with a centered logo/wordmark. The logo pops in at 200 ms, the name slides up at 400 ms; at 1100 ms both images measure the real header logo via `getBoundingClientRect()` and fly to the navbar with `translate(...)` `scale(...)` (FLIP-style) while the red curtain slides off to the top-left, then everything is removed.

```
// Step 3: Exit — fly to navbar header wordmark (1100ms)
nameImg.style.transition = 'transform 0.85s cubic-bezier(0.77,0,0.175,1), opacity 0.65s ease';
nameImg.style.transform = `translate(${nameTx}px, ${nameTy}px) scale(${nameScale})`;
logoImg.style.transition = 'transform 0.85s cubic-bezier(0.77,0,0.175,1), opacity 0.85s ease';
logoImg.style.transform = `translate(${logoTx}px, ${logoTy}px) scale(${logoScale})`;

// Red curtain slides off to top-left corner simultaneously
preloader.style.transform = 'translate(-100%, -100%)';

// Clean up and restore header logo when preloader completes
setTimeout(() => {
  if (headerWordmark) headerWordmark.style.opacity = '1';
  if (logoIcon) logoIcon.style.opacity = '1';
  centerWrap.remove();
  preloader.remove();
}, 950);
```

An SVG stroke-draw keyframe also exists for the logo (currently unused by the sequential loader):

```
.cap-logo-path {
  stroke-dasharray: 600;
  stroke-dashoffset: 600;
  animation: capDraw 0.8s cubic-bezier(0.25, 1, 0.5, 1) forwards;
}
@keyframes capDraw { to { stroke-dashoffset: 0; } }
```

02 — Hero intro: grid lines + character wave

`_setupHeroIntro()` · line 925

Where: Hero section, right after the preloader exits.

How: (a) The vertical hairline SVGs get `scaleX(0) → scaleX(1)` appended to their existing rotation matrix, staggered 80 ms apart. (b) Every large hero text is split into per-character spans inside overflow-hidden word wrappers; each character rises from `translateY(110%)` with a 20 ms-per-char cascade.

```
// 1. Vertical grid lines reveal
svgs.forEach((el, idx) => {
  const originalTransform = el.style.transform || '';
  el.style.transformOrigin = '0 0';
  el.style.transform = originalTransform + ' scaleX(0)';
  el.style.transition = 'transform 1.6s cubic-bezier(0.16, 1, 0.3, 1)';
  setTimeout(() => {
    el.style.transform = originalTransform + ' scaleX(1)';
  }, 100 + idx * 80);
});

// 2. Split-text char stagger wave — per-char initial state
s.style.display = 'inline-block';
s.style.opacity = '0';
s.style.transform = 'translateY(110%)';
s.style.transition = 'transform 0.8s cubic-bezier(0.215, 0.61, 0.355, 1), opacity 0.8s ease';

// cascade in
setTimeout(() => {
  allChars.forEach((cs, cIdx) => {
    setTimeout(() => {
      cs.style.opacity = '1';
      cs.style.transform = 'translateY(0)';
    }, cIdx * 20);
  });
}, 250);
```

03 — Hero headline masks + scroll-out

`_setupHeroTitle()` · line 1000

Where: The 100px headline “India’s Leading / Edu-Infra Asset / Management Company”.

How: Rebuilds the `<br>`-separated span into per-line overflow masks; characters enter from `translateY(115%) rotate(6deg)`, timed (base delay 1150 ms) so the reveal is visible after the preloader curtain leaves. A GSAP ScrollTrigger scrub then makes each line drift up and fade on scroll — lower lines faster, for a parallax exit.

```

// entry state per character
c.style.cssText =
  'display:inline-block; opacity:0;' +
  'transform:translateY(115%) rotate(6deg); transform-origin:0% 100%;' +
  'transition: transform 1.05s cubic-bezier(0.19, 1, 0.22, 1), opacity 0.5s ease;';

// staggered entrance: line by line, chars cascading within each line
lineEls.forEach((L, li) => {
  L.chars.forEach((c, ci) => {
    setTimeout(() => {
      c.style.opacity = '1';
      c.style.transform = 'translateY(0) rotate(0deg)';
    }, 1150 + li * 140 + ci * 20);
  });
});

// scroll-linked parallax exit - lower lines leave faster for depth
lineEls.forEach((L, li) => {
  window.gsap.to(L.lineInner, {
    y: -(40 + li * 30), opacity: 0, ease: 'none',
    scrollTrigger: {
      trigger: document.getElementById('cap-viewport'),
      start: 'top top',
      end: () => '+' + window.innerHeight * 0.8,
      scrub: true
    }
  });
});

```

04 — Water-flow dot grid

`_setupDotGrid()` · line 718

`<canvas id="cap-dotgrid">` · line 455

Where: Canvas overlaying the hero image.

How: A ~130,000-dot grid rendered by writing pixels into a single `ImageData` per rAF frame (with a sine lookup table for speed — direct `fillRect` calls ran at slideshow speed). Each dot eases toward a target displacement composed of: an ambient sine “shimmer”, a smoothstep bulge + directional drag around the spring-smoothed cursor, and gaussian ripple rings shed by fast strokes or clicks — so the cursor reads like a hand moving through water. The loop skips all physics/painting while the hero is off-screen.

```
// Spring-smoothed cursor position (organic tracking)
this._dgVX += (P.x - smoothX) * 0.07; this._dgVY += (P.y - smoothY) * 0.07;
this._dgVX *= 0.78;                    this._dgVY *= 0.78;
smoothX += this._dgVX;                  smoothY += this._dgVY;

// 1. surface bulge + directional drag around the cursor
const dx = d.cx - smoothX, dy = d.cy - smoothY, dsq = dx*dx + dy*dy;
if (cursorGlow > 0.005 && dsq <= currentR2) {
  const dist = Math.sqrt(dsq) || 0.5;
  let t = 1 - dist / currentR;
  t = t * t * (3 - 2 * t);                // smoothstep falloff
  tx += (dx / dist) * BULGE * t * cursorGlow + dragX * t * cursorGlow;
  ty += (dy / dist) * BULGE * t * cursorGlow + dragY * t * cursorGlow;
  a = Math.max(a, baseA + (activeA - baseA) * t * cursorGlow);
}

// 2. ripple rings — a gaussian crest passing through the dot
const crest = Math.exp(-(off * off) / (2 * g.sigma * g.sigma));
const push = g.amp * crest;
tx += (gx / gd) * push;
ty += (gy / gd) * push;
a = Math.min(0.95, a + crest * (g.amp / 22) * 0.5); // the crest glints

// ease toward the target — this is the water smoothness
d.ox += (tx - d.ox) * 0.09;
d.oy += (ty - d.oy) * 0.09;
```

Interaction sources — fast strokes shed wake ripples; clicks drop a two-crest “stone” ripple:

```
// fast strokes shed small wake ripples behind the cursor
if (speed > 550 && now - lastWake > 110) {
  rings.push({ x: nx, y: ny, r: 10, speed: 3.2, amp: 7, sigma: 22 });
}

// click: a dropped-stone ripple — main crest + a softer follower
rings.push({ x: cx, y: cy, r: 6, speed: 5.2, amp: 22, sigma: 30 });
rings.push({ x: cx, y: cy, r: -34, speed: 5.2, amp: 12, sigma: 24 });
```

05 — Text scramble (nav + portfolio)

`_setupNavScramble()` · line 892

`scrambleText()` · line 2741

Where: Header nav links (ABOUT US, PROJECTS, TEAM, CONTACT US); city filters and school names in the Portfolio section.

How: On hover, a 30–32 ms interval rewrites each character with random glyphs from

'ABCDEFGHIJKLMNOPQRSTUVWXYZ#%&*<>/', locking in the real character left-to-right as a frame counter passes its index.

```
el.addEventListener('mouseenter', () => {
  let frame = 0;
  clearInterval(iv);
  iv = setInterval(() => {
    el.textContent = target.split('').map((c, i) => {
      if (c === ' ') return ' ';
      if (i < frame / 2.4) return target[i]; // resolved
      return chars[Math.floor(Math.random() * chars.length)]; // scrambling
    }).join('');
    frame++;
    if (frame / 2.4 >= target.length) { clearInterval(iv); el.textContent = target; }
  }, 32);
});
```

The portfolio version wraps characters in spans (`wrapInCharSpans`) and calls `alignLogo()` every frame so the red interlocking-C logo stays glued to the end of the active city name while it scrambles:

```
interval = setInterval(() => {
  charSpans.forEach((cs, index) => {
    const char = originalText[index];
    if (char === ' ' || char === '\n' || char === ' ') return;
    cs.textContent = (index < step)
      ? char
      : chars[Math.floor(Math.random() * chars.length)];
  });
  alignLogo(); // keep the C logo tracking during scramble frames
  step += 1;
  if (step > originalText.length) { clearInterval(interval); /* restore */ }
}, 30);
```

06 — Generic scroll reveals

CSS · lines 46-106

_setupScrollReveals() · line 1092

Where: Section headings, stat numbers, and body paragraphs across the whole page.

How: JS classifies elements heuristically (font size, text length, position) and tags them with reveal classes; an IntersectionObserver adds `.cap-revealed` once at 12% visibility. All motion is pure CSS, gated behind `prefers-reduced-motion: no-preference`.

```
.cap-scroll-reveal {
  opacity: 0;
  transform: translateY(40px);
  transition: opacity 0.75s cubic-bezier(0.22, 1, 0.36, 1),
             transform 0.75s cubic-bezier(0.22, 1, 0.36, 1);
}
.cap-scroll-reveal.cap-revealed { opacity: 1; transform: translateY(0); }
.cap-scroll-reveal.cap-delay-1 { transition-delay: 0.1s; } /* ...through -5 */

.cap-slide-left { opacity: 0; transform: translateX(-60px); } /* variant */
.cap-scale-reveal { opacity: 0; transform: scale(0.88); } /* stats */
.cap-clip-reveal > * { transform: translateY(100%); } /* masked */
```

```
const io = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      entry.target.classList.add('cap-revealed');
      io.unobserve(entry.target);
    }
  });
}, { threshold: 0.12, rootMargin: '0px 0px -60px 0px' });

// STAT NUMBERS — large metric text ($500 Mn+, 135 Acres, 16 Assets, 75 Years+)
statNums.forEach((el, i) => {
  addReveal(el, 'cap-scale-reveal', 'cap-delay-' + Math.min(i + 1, 5));
  io.observe(el);
});
```

07 — Lenis smooth scroll + GSAP ticker

line 1167

Where: Global — the foundation every scroll-scrubbed animation rides on.

How: Lenis inertial scrolling is synced into GSAP's ticker so ScrollTrigger scrubs stay frame-accurate. Nav and menu links glide to sections via `lenis.scrollTo(target)`.

```
this._lenis = new window.Lenis({
  duration: 1.2,
  easing: (t) => Math.min(1, 1.001 - Math.pow(2, -10 * t)), // OCI style easing
});
this._lenis.on('scroll', window.ScrollTrigger.update);
window.gsap.ticker.add((time) => { this._lenis.raf(time * 1000); });
window.gsap.ticker.lagSmoothing(0);
```

08 — Hero parallax zoom

`_setupHeroZoom()` · line 1388

Where: The hero photo wrapper.

How: GSAP scrubs the whole hero block to `scale(1.15)` over the first 1200 px of scroll, anchored at bottom-center so the horizon stays planted.

```
heroWrapper.style.transformOrigin = '50% 100%'; // zoom from bottom center
window.gsap.to(heroWrapper, {
  scale: 1.15,
  ease: 'none',
  scrollTrigger: {
    trigger: document.body,
    start: 'top top',
    end: '1200px',
    scrub: true
  }
});
```

09 — Line-split paragraph reveals

`_setupSplitText()` · line 1414

Where: All long body paragraphs (takes over from the generic reveal on those elements).

How: SplitType splits each paragraph into rendered lines; every line is wrapped in an overflow-hidden mask, set to `yPercent: 100`, and GSAP staggers them up the first time the paragraph enters the viewport.

```
const split = new window.SplitType(el, { types: 'lines' });

split.lines.forEach(line => {
  const wrapper = document.createElement('div');
  wrapper.style.overflow = 'hidden';
  wrapper.style.display = 'inline-block';
  line.parentNode.insertBefore(wrapper, line);
  wrapper.appendChild(line);
  window.gsap.set(line, { yPercent: 100 });
});

window.ScrollTrigger.create({
  trigger: el,
  start: 'top 90%',
  onEnter: () => {
    window.gsap.to(split.lines, {
      yPercent: 0, duration: 1.2, ease: 'power4.out', stagger: 0.08
    });
  }
});
```


10 — Odometer stat counters

`_setupStatCounters()` · line 2196

Where: Every large stat figure — \$500 Mn+, 135 Acres, 16 Assets, 75 Years+, 10 Years (leaf text nodes ≥ 55 px).

How: Each digit becomes a masked 1em-tall window over a column containing 0–9 twice. On first visibility the column snaps to 0 (transition disabled + forced reflow), then rolls to `-(10 + digit)em` so every digit passes one full cycle before landing, staggered 90 ms per digit.

```
// build: mask + rolling column per digit
const mask = document.createElement('span');
mask.style.cssText = 'display:inline-block; overflow:hidden; height:1em; line-height:1em;';
const col = document.createElement('span');
col.style.cssText = 'display:block; will-change:transform; transform:translateY(-' + digit + 'em)';
for (let r = 0; r < 2; r++)
  for (let d = 0; d <= 9; d++) { /* append 0-9 cells, each 1em tall */ }

// roll on first visibility
const io = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (!entry.isIntersecting) return;
    io.disconnect();
    rolls.forEach(({ col, digit }, i) => {
      col.style.transition = 'none';
      col.style.transform = 'translateY(0)';
      void col.offsetHeight; // reflow so the jump to 0 doesn't animate
      col.style.transition = 'transform 1.5s var(--cap-ease-expo) ' + (i * 0.09) + 's';
      col.style.transform = 'translateY(-' + (10 + digit) + 'em)';
    });
  });
}, { threshold: 0.4 });
```

11 — Whitespace staircase reveal

`markup` · line 469

`_setupReveal()` · line 2281

Where: The white staircase notch cutting into the red stats band, under “135 Acres”.

How: Four white bars (widths 25% → 100%) pre-collapsed with `scaleX(0)` inside a clipped container. Scroll position maps to a 0–1 progress; each bar gets a staggered slice of it (bottom first, 0.15 delay steps over a 0.55 window), eased with `easeInOutQuad` and applied as a left-anchored `scaleX`. Because it is scroll-scrubbed, it also plays backwards when scrolling up. The “75 Years+” stat fades/slides in during the tail.

```

<div id="cap-reveal" style="position:absolute; left:0; top:1715px; width:728px;
  height:440px; z-index:6; pointer-events:none; overflow:hidden;">
  <div class="cap-stair" style="... top:0; width:25%; height:110px;
    background:#fff; transform-origin:left; transform:scaleX(0);"></div>
  <div class="cap-stair" style="... top:110px; width:50%; ..."></div>
  <div class="cap-stair" style="... top:220px; width:75%; ..."></div>
  <div class="cap-stair" style="... top:330px; width:100%; ..."></div>
</div>

```

```

this._onRevealScroll = () => {
  const r = reveal.getBoundingClientRect();
  const vh = window.innerHeight;
  // 0 when the reveal top is near the bottom of viewport, 1 when well above it
  const p = clamp((vh * 0.85 - r.top) / (vh * 0.85));

  const span = 0.55, stagger = 0.15;
  stairs.forEach((stair, i) => {
    const delay = (N - 1 - i) * stagger; // bottom stair starts first
    const stairProgress = clamp((p - delay) / span);
    stair.style.transform = 'scaleX(' + easeInOutQuad(stairProgress) + ')';
  });

  if (stat75) { // "75 Years+" fades in at the tail
    const fp = clamp((p - 0.45) / 0.4);
    stat75.style.opacity = String(fp);
    stat75.style.transform = 'translateY(' + (30 * (1 - fp)) + 'px)';
  }
};
window.addEventListener('scroll', this._onRevealScroll, { passive: true });

```

12 — Journey: S-curve draw + phase slider

`_setupJourneyStory()` · line 1920

Where: “Our Journey” section (frame top $\approx$ 2959–3712).

How (a): S-curve line draw — the classic dasharray trick, scrubbed by scroll:

```
const len = path.getTotalLength();
path.style.strokeDasharray = len;
window.gsap.fromTo(path, { strokeDashoffset: len }, {
  strokeDashoffset: len * 2,
  ease: 'none',
  scrollTrigger: { trigger: svg, start: 'top 82%', end: 'bottom 45%', scrub: true }
});
```

How (b): Phase transitions (Foundation → Growth → Expansion) — a skewed white gradient band sweeps diagonally across the section; the backdrop/building classes are swapped mid-wipe (450 ms in) so the change hides under the sweep, and the new photo settles with a GSAP scale/opacity pop:

```
// 1. Start sweep diagonal wipe
window.gsap.set(sweep, { transform: 'skewX(-20deg) translateX(-110%)' });
window.gsap.to(sweep, {
  transform: 'skewX(-20deg) translateX(110%)',
  duration: 0.95, ease: 'power3.inOut'
});

// 2. Perform swap at mid-wipe (450ms)
setTimeout(() => {
  backdrop.className = JOURNEY_DATA[targetIdx].backdropClass;
  building.className = JOURNEY_DATA[targetIdx].imageClass;
  renderSlide(targetIdx, yearValOpt);
  window.gsap.fromTo(building, { opacity: 0.2, scale: 0.95 },
    { opacity: 1, scale: 1, duration: 0.8, ease: 'power2.out' });
}, 450);

// 3. Reset sweep overlay at the end
setTimeout(() => {
  window.gsap.set(sweep, { transform: 'skewX(-20deg) translateX(-110%)' });
  isTransitioning = false;
}, 1000);
```

How (c): Year dots and labels — rebuilt per phase and staggered in; dots have a springy back-ease hover and an active glow state:

```
window.gsap.fromTo(animatedElements,
  { opacity: 0, y: 15 },
  { opacity: (el) => el.style.color === 'rgb(0,0,0)' ? 0.95 : 1,
    y: 0, duration: 0.65, stagger: 0.08, ease: 'power2.out' });
```

```
.cap-journey-dot {
  transition: transform 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275),
    background 0.3s ease, border-color 0.3s ease;
}
.cap-journey-dot:hover { transform: translate(-10px, -10px) scale(1.3); }
.cap-journey-dot.cap-dot-active {
  background: rgb(209, 32, 47); border-color: #fff;
  transform: translate(-10px, -10px) scale(1.35);
  box-shadow: 0 0 20px rgba(255, 255, 255, 0.8);
}
```

13 — Dither-resolve photo entries

`_setupDitherEntries()` · line 1692

Where: The journey full-width backdrop and the journey building photo.

How: A canvas overlay is painted with a red-duotone 4×4 Bayer ordered dither of the photo (downsampled to 190 px wide, so the halftone is chunky when stretched). It sits at full opacity until the photo scrolls into view, then fades out — “resolving” the image to full colour.

```
const bayer = [[0,12,3,15],[8,4,11,7],[2,14,1,13],[10,6,9,5]];
for (let y = 0; y < h; y++) {
  for (let x = 0; x < w; x++) {
    const i4 = (y * w + x) * 4;
    const gray = 0.299*px[i4] + 0.587*px[i4+1] + 0.114*px[i4+2];
    const light = gray > ((bayer[y % 4][x % 4] + 0.5) / 16) * 255;
    px[i4] = light ? 255 : 209; // light → warm white
    px[i4+1] = light ? 235 : 32; // dark → Cappella red
    px[i4+2] = light ? 235 : 47;
  }
}
ctx.putImageData(imgData, 0, 0);

// Dithered until the photo scrolls in, then resolve to full colour
canvas.style.opacity = '1';
canvas.style.transition = 'opacity 0.9s var(--cap-ease-power) 0.2s';
const io = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (!entry.isIntersecting) return;
    io.disconnect();
    canvas.style.opacity = '0';
  });
}, { threshold: 0.3 });
io.observe(el);
```

14 — Portfolio WebGL dither hover

`_setupPortfolioInteraction()` · line 2323

`createWebGLDither()` · line 2478

Where: The four school cards in “Our Portfolio”.

How: A real GLSL fragment shader per card (architecture mirrors the OCI website’s dither.js). Cards render as red duotone Bayer dither by default; hovering tweens uniform `u_dither → 0` and `u_zoom → 1.05` on `requestAnimationFrame` so the true photo resolves with a slight zoom. The canvas is downsampled to ~35% and stretched by CSS so each 4×4 Bayer cell covers ~10 screen pixels (chunky halftone).

```
/* fragment shader */
precision mediump float;
varying vec2 v_uv;
uniform sampler2D u_tex;
uniform float u_dither;      // 0=colour, 1=dither
uniform float u_zoom;        // 1.0 default, 1.05 hovered
uniform float u_bayer[16];   // 4x4 Bayer matrix values

void main() {
    vec2 uv = (v_uv - 0.5) / u_zoom + 0.5;          /* zoom from centre */
    vec4 col = texture2D(u_tex, uv);
    float lum = dot(col.rgb, vec3(0.299, 0.587, 0.114)); /* luminance */

    /* 4x4 Bayer threshold lookup on pixel coords (branch chain for GLSL ES 1.0) */
    float threshold = (thr + 0.5) / 16.0;

    vec3 dark  = vec3(0.820, 0.125, 0.184); /* #D1202F Cappella red */
    vec3 light = vec3(1.000, 0.941, 0.941); /* #FFF0F0 warm white */
    vec3 dithered = lum > threshold ? light : dark;

    gl_FragColor = vec4(mix(col.rgb, dithered, u_dither), col.a);
}
```

```
// rAF tween driving the uniforms (mimics GSAP power2.out)
const ease = t => 1 - Math.pow(1 - t, 3);
const tween = (fromD, toD, fromZ, toZ, dur) => {
    const start = performance.now();
    const step = now => {
        const et = ease(Math.min((now - start) / dur, 1));
        ditherAmt = fromD + (toD - fromD) * et;
        zoom      = fromZ + (toZ - fromZ) * et;
        render();
        if (et < 1) rafId = requestAnimationFrame(step);
    };
    rafId = requestAnimationFrame(step);
};

return {
    showOriginal() { tween(ditherAmt, 0.0, zoom, 1.05, 700); }, // hover
    showDither()   { tween(ditherAmt, 1.0, zoom, 1.00, 700); }, // leave
};
```

Also in this method: city filtering (non-matching cards drop to `opacity:0; translateY(30px) scale(0.95)`), a two-column scroll parallax, a corner arrow button sliding in on hover, and the scramble-

tracking logo alignment (section 05).

```
// two-column scroll parallax - left column drifts up, right column down
const centerOffset = (scrollY + window.innerHeight / 2 - 4400 * s) / s;
const offsetVal = centerOffset * 0.12;
cards.forEach(c => {
  const isCol1 = (c.id === 1 || c.id === 3);
  c.parallaxY = isCol1 ? -offsetVal : offsetVal;
});

// arrow button (bottom-right corner slides in on hover)
'transition:transform 0.45s cubic-bezier(0.25,1,0.5,1), opacity 0.45s ease',
'transform:translate(100%,100%)', 'opacity:0'
// hover → translate(0,0), opacity 1
```

15 — “Cappella Edge” accordion

`_setupAccordion()` · line 2908

Where: The four-row accordion in “The Cappella Edge” section.

How: Rows are absolutely positioned (baked Figma frame), so opening item *i* recomputes every `top` below it (`shift += openH`) with a shared `0.65s cubic-bezier(0.76, 0, 0.24, 1)` transition. Per item: an OCI-style hover roll (title duplicated inside an overflow mask), a plus icon that rotates 135° into an ×, a masked description rising from `translateY(110%)`, and a dither flash — the section photo briefly shows its Bayer-dithered canvas and scales to 1.03 on every open/close.

```
const layout = () => {
  let shift = 0;
  items.forEach((item, i) => {
    const isOpen = i === activeIndex;
    const top = item.closedTop + shift;
    item.titleEl.style.top = top + 'px';
    item.iconContainer.style.top = (top + ICON_OFFSET) + 'px';

    // OCI-exact GSAP rotation: 135deg open (x), 0deg closed
    window.gsap.to(item.iconContainer, {
      rotation: isOpen ? 135 : 0,
      duration: 0.8, ease: 'power4.inOut', overwrite: true
    });

    item.descWrap.style.top = (top + DESC_OFFSET) + 'px';
    if (isOpen) {
      item.inner.style.transitionDelay = '0.15s, 0.15s';
      item.inner.style.transform = 'translateY(0)'; // masked desc rises in
      item.inner.style.opacity = '1';
      shift += item.openH; // push rows below down
    } else {
      item.inner.style.transform = 'translateY(110%)';
      item.inner.style.opacity = '0';
    }
    item.setDividerTop(item.closedDividerTop + shift);
  });
};
```

```
// Dithered flash on the section image whenever the accordion changes
flashImage = () => {
  canvas.style.opacity = '1'; // Bayer-dithered overlay
  edgeImg.style.transform = 'scale(1.03)';
  clearTimeout(flashT);
  flashT = setTimeout(() => {
    canvas.style.opacity = '0';
    edgeImg.style.transform = 'scale(1)';
  }, 400);
};
```

16 — Footer: curtain reveal + staged rise + live clock

`_setupFooterAnimations()` · line 1466

`.cap-ft-link` CSS · line 434

Where: The red footer.

How: Three layers. (a) **Curtain reveal** — the footer becomes an `overflow:hidden` window; all content starts at `y:-420` and scrubs to 0 as the footer scrolls in, so it reads as a fixed surface being uncovered. Stray Figma nodes tagged `data-original-top` are pulled into the slide set so the curtain doesn't shear the footer apart. (b) **Staged fade-in** — the big C logo wipes in via clip-path; everything else fades with a delay keyed to its column position. (c) A **live Hyderabad clock** ticks every second; footer links get an underline wipe; the email arrow nudges on hover.

```
// (a) curtain scrub
window.gsap.fromTo(slide, { y: -420 }, {
  y: 0, ease: 'none',
  scrollTrigger: { trigger: footer, start: 'top bottom',
    end: 'bottom bottom', scrub: true }
});

// (b) staged rise — delay keyed to column position
const delay = 0.15 + (b.left / 1440) * 0.4 + Math.max(0, relTop / 530) * 0.15;
if (el === bigLogo) {
  el.style.clipPath = 'inset(0 100% 0 0)';
  el.style.transition = 'clip-path 1.1s var(--cap-ease-expo) 0.1s';
  staged.push(() => { el.style.clipPath = 'inset(0 0% 0 0)'; });
} else {
  el.style.opacity = '0'; // opacity-only: the curtain owns the transform channel
  el.style.transition = 'opacity 0.85s var(--cap-ease-expo) ' + delay + 's';
  staged.push(() => { el.style.opacity = '1'; });
}

// (c) live local-time line
const tickClock = () => {
  clock.textContent = 'HYDERABAD — ' +
    new Date().toLocaleTimeString('en-IN',
      { timeZone: 'Asia/Kolkata', hour12: false }) + ' IST';
};
this._clockIv = setInterval(tickClock, 1000);
```

```
/* underline wipe on footer links */
.cap-ft-link::after {
  content: ''; position: absolute; left: 0; bottom: -3px;
  width: 100%; height: 1px; background: rgb(255, 255, 255);
  transform: scaleX(0); transform-origin: left;
  transition: transform 0.45s var(--cap-ease-power);
}
.cap-ft-link:hover::after { transform: scaleX(1); }
```


17 — Operators logo marquee

`_setupOperatorsMarquee()` · line 1603

Where: The operator logos section — two bands scrolling in opposite directions.

How: The static Figma logo grid is hidden; logos are cloned into a flex track repeated enough times to cover the width. A shared rAF loop translates each track, wrapping via modulo on the sequence width. Speed eases between 34 px/s and 0 on hover (soft pause/resume), logos nudge up 6 px on hover, bands fade in on first visibility, and the loop skips work while both bands are off-screen.

```
const repeats = Math.max(3, Math.ceil((1440 + seqW) / seqW) + 1);
band.addEventListener('mouseenter', () => { row.targetSpeed = 0; });
band.addEventListener('mouseleave', () => { row.targetSpeed = 34; });

// Shared drive loop — skips work while both bands are off-screen
const step = (now) => {
  const dt = Math.min((now - lastT) / 1000, 0.05);
  lastT = now;
  rows.forEach(row => {
    const r = row.band.getBoundingClientRect();
    if (r.bottom < -100 || r.top > window.innerHeight + 100) return;
    row.speed += (row.targetSpeed - row.speed) * 0.06; // eased pause/resume
    row.x = (row.x + row.dir * row.speed * dt) % row.seqW;
    const offset = row.dir === -1 ? row.x : row.x - row.seqW;
    row.track.style.transform = 'translateX(' + offset.toFixed(2) + 'px)';
  });
  this._mqRaf = requestAnimationFrame(step);
};
this._mqRaf = requestAnimationFrame(step);
```

18 — Menu overlay

CSS · lines 261-431

`_setupMenuOverlay()` · line 1762

Where: Floating hamburger button (appears after ~180 px of scroll) and the slide-in side panel.

How: Entirely CSS class-toggle driven — adding `.cap-menu-open` slides the panel in from `translateX(102%)`, fades the backdrop, and the big links rise out of overflow masks with per-index transition delays set in JS. Rows and the email field follow with their own delays; links indent 18 px on hover.

```
#cap-menu-panel {
  transform: translateX(102%);
  transition: transform 0.75s var(--cap-ease-power);
}
.cap-menu-open #cap-menu-panel { transform: translateX(0); }

.cap-menu-links .cap-menu-mask { overflow: hidden; }
.cap-menu-links a {
  transform: translateY(115%);
  transition: transform 0.8s var(--cap-ease-expo),
    padding-left 0.45s var(--cap-ease-power);
}
.cap-menu-open .cap-menu-links a { transform: translateY(0); }
.cap-menu-links a:hover { padding-left: 18px; }

.cap-menu-row { opacity: 0; transform: translateY(16px);
  transition: opacity 0.7s var(--cap-ease-expo), transform 0.7s var(--cap-ease-expo); }
.cap-menu-open .cap-menu-row { opacity: 1; transform: translateY(0); }
```

```
// Staggered rise for the big links (transition-delay set per index)
const links = [...overlay.querySelectorAll('.cap-menu-links a')];
links.forEach((a, i) => { a.style.transitionDelay = (0.12 + i * 0.07) + 's, 0s'; });

const open = () => {
  overlay.classList.add('cap-menu-open');
  if (this._lenis) this._lenis.stop();
  document.body.style.overflow = 'hidden';
};
```

19 — Film grain

CSS · lines 158–180

`_setupGrain()` · line 1882

Where: Fixed overlay across the entire page (z-index 95).

How: An inline-SVG `feTurbulence` noise tile at 5% opacity with `mix-blend-mode: overlay`, jittered by a 7-step `steps()` keyframe loop so the noise “boils” like film grain rather than sliding.

```
#cap-grain {
  position: fixed; inset: -120px; z-index: 95;
  pointer-events: none; opacity: 0.05; mix-blend-mode: overlay;
  background-image: url("data:image/svg+xml,...feTurbulence type='fractalNoise'
                        baseFrequency='0.85' numOctaves='2'...");
}
@media (prefers-reduced-motion: no-preference) {
  #cap-grain { animation: capGrain 0.7s steps(7) infinite; }
}
@keyframes capGrain {
  0%   { transform: translate(0, 0); }
  14%  { transform: translate(-38px, 22px); }
  28%  { transform: translate(24px, -44px); }
  42%  { transform: translate(-52px, -18px); }
  56%  { transform: translate(40px, 36px); }
  70%  { transform: translate(-20px, 48px); }
  84%  { transform: translate(46px, -12px); }
  100% { transform: translate(0, 0); }
}
```

Inactive systems & accessibility notes

Inactive: `_setupMetaLabels()` (line 1889) — editorial [01 — THE CAPPELLA EDGE] labels that fade in per section; the call is commented out at line 706. `_setupJourneyDISABLED()` (line 2261) — an older scroll-linked journey phase crossfade, superseded by the phase slider (section 12).

Accessibility & performance: Nearly every motion system checks `prefers-reduced-motion: reduce` and degrades to static content. The heavy rAF loops (dot grid, marquee) idle when their section is off-screen. The dot grid renders through a single `ImageData` pixel buffer with a sine lookup table to hold 60 fps at ~130k dots.